



INSTITUTO FEDERAL GOIANO
CAMPUS URUTAÍ
NÚCLEO DE INFORMÁTICA
ESPECIALIZAÇÃO EM INTELIGÊNCIA ARTIFICIAL APLICADA A DADOS CORPORATIVOS

LEONARDO MAXIMINO BERNARDO

**ACOLHEMENTE ESCOLAR PB: MOTOR DE INFERÊNCIA LÓGICO
PARA APOIO À GESTÃO ESCOLAR NA PRIORIZAÇÃO
RESPONSÁVEL DE ACOLHIMENTO EM SAÚDE MENTAL**

Urutaí, maio de 2026

INSTITUTO FEDERAL GOIANO
CAMPUS URUTAÍ
Núcleo de Informática
Pós-Graduação *Lato Sensu* em Inteligência Artificial Aplicada a Dados Corporativos

Trabalho de Conclusão da Trilha de Aprendizagem I
Inteligência Artificial Simbólica e Busca

AcolheMente Escolar PB

Motor de Inferência Lógico para Apoio à Gestão Escolar
na Priorização Responsável de Acolhimento em Saúde Mental

Autor: Leonardo Maximino Bernardo
Orientação: Corpo docente da Trilha I — IF Goiano

Sumário

1	Apresentação do Aluno e Contexto	2
2	Introdução e Definição do Problema	2
3	Metodologia e Escolha da Técnica de IA	3
3.1	Arquitetura de Dados, Proveniência e Governança	4
3.2	Representação do Conhecimento: 7 Variáveis Proposicionais	5
3.3	Base de Conhecimento: 6 Regras R1–R6	6
4	Implementação Acadêmica e Motores Equivalentes	6
4.0.1	Engenharia de Software e Garantia de Qualidade	6
4.0.2	Nota sobre R5 e subsunção lógica	8
4.0.3	Nota metodológica de governança	8
5	Resultados e Comparações	9
5.0.1	Comparação: <i>Baseline</i> Manual vs. Motor Lógico	10
5.1	Explicabilidade	11
5.2	Discussão Crítica	12
6	Perspectiva de Evolução do Trabalho	12
7	Conclusões	13
8	Referências Bibliográficas	13
	Apêndice A — Símbolos Proposicionais do AcolheMente	15
	Apêndice B — Regras Lógicas e CNF	18
	Apêndice C — Motor de Inferência por Resolução	20
	Apêndice D — Cenários Sintéticos de Validação	21
	Apêndice E — Grafo de Explicabilidade	23
	Apêndice F — Gerador de Tabelas de Resultados	25
	Apêndice G — Declaração de Uso de Inteligência Artificial	27

1 Apresentação do Aluno e Contexto

Leonardo Maximino Bernardo é professor de Física da rede pública estadual da Paraíba desde 2020, com mais de 15 anos de experiência em docência nos níveis médio e superior. Doutor em Ciência e Engenharia de Materiais pela Universidade Federal da Paraíba, sua trajetória acadêmica combina modelagem computacional avançada e uma crescente atuação em ciência de dados e inteligência artificial.

Ao longo de sua formação, atuou como pesquisador na UFPB, desenvolvendo rotinas de cálculo e algoritmos para análise de grandes volumes de dados de simulação numérica de solidificação de ligas metálicas, extraíndo padrões quantitativos para publicações acadêmicas. Essa experiência consolidou habilidades de pensamento analítico, resolução de problemas complexos e comunicação científica que informam diretamente sua prática docente e sua abordagem em projetos de inteligência artificial.

Atualmente, além da docência em Física, cursa Pós-Graduação *Lato Sensu* em Inteligência Artificial Aplicada pelo Instituto Federal Goiano e possui formação em Ciência de Dados pelo Unipê. Sua *stack* tecnológica inclui Python, JavaScript, SQL, *frameworks* como Django e React, e ferramentas de análise como Pandas, Scikit-learn e NetworkX. Possui certificações em desenvolvimento *back-end* com Node.js, Harvard CS50 e aceleração global de desenvolvimento de software.

O presente trabalho nasce da intersecção entre sua vivência como professor de escola pública estadual na Paraíba — onde testemunha diariamente demandas crescentes de acolhimento em saúde mental de adolescentes — e seu domínio técnico em modelagem computacional e lógica formal. A escolha do domínio de saúde mental escolar reflete o compromisso de aplicar inteligência artificial de forma ética, explicável e governada em um contexto de vulnerabilidade social, onde erros tecnológicos podem ter consequências humanas graves.

2 Introdução e Definição do Problema

A saúde mental de adolescentes no Brasil atravessa um momento crítico. Dados da Pesquisa Nacional de Saúde do Escolar — PeNSE 2024 (IBGE, 2024) — revelam que parcela significativa dos estudantes de 13 a 17 anos reporta sentimentos persistentes de tristeza, solidão e desesperança, com índices particularmente preocupantes na região Nordeste e, especificamente, no estado da Paraíba. O relatório da Organização Mundial da Saúde (WHO, 2022) confirma essa tendência global: transtornos mentais respondem por uma proporção crescente da carga de doença entre jovens, e a maioria dos casos permanece sem qualquer forma de acolhimento adequado.

No contexto da escola pública estadual paraibana, a demanda por acolhimento em saúde mental frequentemente supera a capacidade de resposta da equipe pedagógica e psicossocial. Professores, coordenadores e orientadores educacionais enfrentam o desafio de identificar, dentre centenas de estudantes, aqueles que necessitam de atenção prioritária — sem dispor de ferramentas estruturadas para essa tarefa. O resultado é uma dependência excessiva de percepções individuais, sujeitas a viés cognitivo, fadiga decisória e inconsistência entre turnos e unidades escolares. Essa lacuna não é trivial: a ausência de priorização sistemática pode significar que um adolescente em situação de risco grave não receba acolhimento em tempo hábil.

Ao mesmo tempo, a adoção acrítica de tecnologias de inteligência artificial nesse domínio apresenta riscos igualmente sérios. Sistemas baseados em aprendizado de máquina ou IA generativa, embora poderosos em outros contextos, introduzem opacidade decisória incompatível com o rigor ético exigido quando se trata de menores de idade em situação de vulnerabilidade (Jobin; Ienca; Vayena, 2019). A Lei Geral de Proteção de Dados — LGPD (Brasil, 2018) — e o Estatuto da Criança e do Adolescente — ECA (Brasil, 1990) — impõem salvaguardas rigorosas para o tratamento de dados sensíveis de menores, incluindo o direito à explicação de decisões automatizadas. Um modelo de “caixa preta” que classifique ou ranquee estudantes por risco psicológico seria, no mínimo, juridicamente temerário e, no limite, eticamente inaceitável.

Diante desse cenário, o presente trabalho propõe o **AcolheMente Escolar PB**: um Sistema Baseado em Conhecimento que utiliza Lógica Proposicional com Inferência por Resolução para apoiar — e nunca substituir — a gestão escolar na priorização responsável de acolhimento em saúde mental. A escolha por lógica simbólica determinística não é uma limitação técnica, mas uma decisão de *design* ético: garante explicabilidade total, rastreabilidade de cada conclusão e ausência de qualquer linguagem diagnóstica clínica (Russell; Norvig, 2022).

Problema central: Como apoiar a gestão escolar de escolas públicas estaduais da Paraíba na priorização sistemática de acolhimento em saúde mental de adolescentes, utilizando inteligência artificial explicável, sem produzir diagnósticos clínicos, sem violar a LGPD ou o ECA, e sem substituir o julgamento humano profissional?

Objetivo geral: Desenvolver e validar um motor de inferência por resolução, baseado em lógica proposicional e fundamentado em dados oficiais da PeNSE 2024, para apoiar a priorização de acolhimento em saúde mental escolar no estado da Paraíba.

Objetivos específicos: 1. Modelar o domínio em 7 variáveis proposicionais (4 entradas nucleares, 2 entradas contextuais e 1 saída inferida), ancoradas em variáveis da PeNSE 2024. 2. Formalizar 6 regras de inferência em Forma Normal Conjuntiva e implementar motor de resolução determinístico. 3. Validar o motor com cenários sintéticos representativos e comparar com *baseline* manual. 4. Garantir conformidade integral com LGPD, ECA e Programa Saúde na Escola (Brasil, 2007). 5. Documentar explicabilidade, limitações e perspectivas de evolução com rigor acadêmico.

3 Metodologia e Escolha da Técnica de IA

O AcolheMente Escolar PB é, em sua essência, um Sistema Baseado em Conhecimento — uma classe de sistemas de inteligência artificial que codifica explicitamente o conhecimento de especialistas humanos em regras formais e utiliza mecanismos de inferência para derivar conclusões a partir de fatos observados (Brachman; Levesque, 2004). Diferentemente de abordagens estatísticas ou conexionistas, um SBC não “aprende” a partir de dados brutos: opera sobre uma base de conhecimento construída deliberadamente por engenheiros do conhecimento em colaboração com especialistas do domínio.

A técnica específica adotada é a **Lógica Proposicional com Inferência por Resolução**. Trata-se de um formalismo da lógica clássica no qual o conhecimento é representado por proposições atômicas e conectivos lógicos. A inferência é realizada por meio do método de resolução, um procedimento correto e refutacionalmente completo para a lógica proposicional (Genesereth; Nilsson, 1987). O método opera sobre cláusulas em Forma Normal

Conjuntiva: para verificar se uma conclusão A pode ser derivada de uma base de conhecimento KB , adiciona-se a negação da conclusão ($\neg A$) ao conjunto de cláusulas e aplica-se repetidamente a regra de resolução, buscando derivar a cláusula vazia — o que constitui prova por contradição de que $KB \models A$.

Foram consideradas e descartadas alternativas como Lógica de Primeira Ordem (complexidade desnecessária para 7 variáveis binárias), Redes Bayesianas (exigiriam distribuições de probabilidade inestimáveis), Árvores de Decisão (dependem de dados rotulados inexistentes), Redes Neurais (incompatíveis com explicabilidade total) e IA Generativa (sem garantias de determinismo).

Conforme argumentam Russell e Norvig (2022), a escolha da representação do conhecimento deve ser guiada pelas propriedades do domínio, e não pela sofisticação da técnica. No caso do AcolheMente Escolar PB, a lógica proposicional é a ferramenta *correta*, não uma ferramenta *limitada*.

O princípio de *human-in-the-loop* é inegociável. A variável de saída A é um sinal para o profissional humano, não uma decisão final.

3.1 Arquitetura de Dados, Proveniência e Governança

O AcolheMente opera com arquitetura rigorosa de proveniência de dados em três camadas mutuamente exclusivas. A separação não é apenas técnica: é um requisito de governança que impede contaminação cruzada entre fontes de naturezas distintas.

Tier	Fonte	Uso Permitido	Uso Proibido	Justificativa
TIER_A	PeNSE 2024 (IBGE)	EDA, motivação, variáveis	Inferência individual	Pesquisa populacional
TIER_B	Cenários sintéticos	Validação do motor	Representar alunos reais	Controle determinístico
TIER_C	Dataset externo	Benchmark metodológico	Conclusão sobre PB/BR	Sem validade regional

Arquitetura de Proveniência de Dados --- AcolheMente Escolar PB

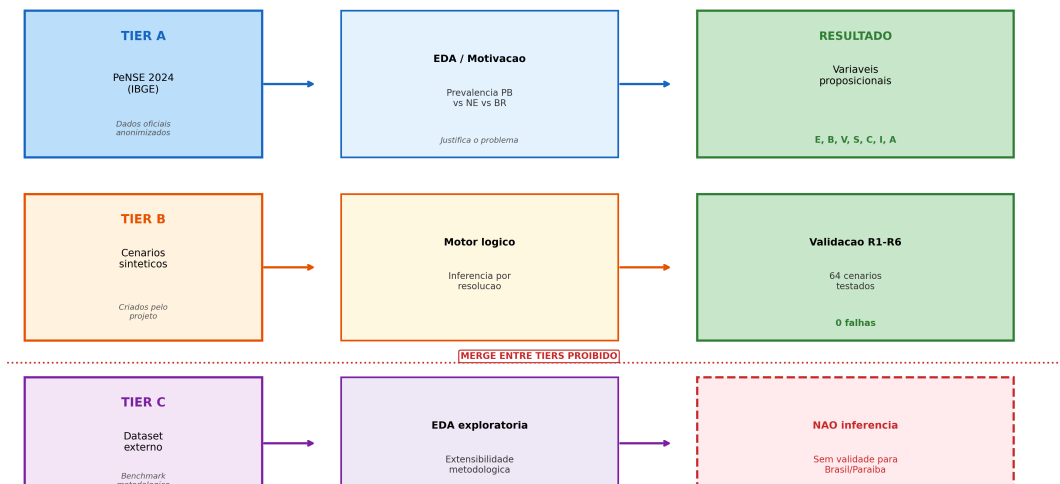


Figura 1: Figura — Fluxo de proveniência de dados: cada tier alimenta exclusivamente seu destino legítimo.

Merge entre tiers é estritamente proibido. Os dados da PeNSE **não alimentam diretamente o motor de inferência** para classificação individual: a PeNSE é pesquisa populacional, não sistema de acompanhamento escolar ativo. O motor é validado por cenários sintéticos determinísticos (TIER_B), enquanto a PeNSE sustenta a justificativa epidemiológica e a modelagem conceitual (TIER_A).

A LGPD (Brasil, 2018) classifica dados de saúde de menores como dados pessoais sensíveis. O AcolheMente atende *by design*: não processa dados pessoais identificáveis, opera sobre indicadores agregados anonimizados, toda conclusão é rastreável e não produz decisões automatizadas sobre indivíduos. O ECA (Brasil, 1990) veda qualquer forma de rotulagem de crianças e adolescentes.

3.2 Representação do Conhecimento: 7 Variáveis Proposicionais

A modelagem do domínio resultou em 7 variáveis proposicionais — 4 entradas nucleares, 2 entradas contextuais e 1 saída inferida:

Var	Tipo	Semântica Operacional	Fontes PeNSE
E	Nuclear	Sofrimento emocional recorrente	B12004, B12005, B12007
B	Nuclear	Baixo apoio socioafetivo percebido	B12003, B07004
V	Nuclear	Indicador crítico de desvalor da vida	B12008
S	Nuclear	Sinal autorreferido de autoagressão	B12009
A	Nuclear	Acolhimento humano prioritário (saída)	Inferida
C	Contextual	Contexto comportamental agravante	B03010C, B03006B
I	Contextual	Insuficiência institucional de resposta	E01P60, E01P117

As variáveis contextuais **nunca inferem A isoladamente** — este é um *guardrail* lógico fundamental.

O código-fonte completo da definição formal dos símbolos, incluindo tipos, descrições e rastreabilidade PeNSE, está disponível no Apêndice A.

3.3 Base de Conhecimento: 6 Regras R1–R6

Regra	Forma Implicativa	CNF
R1	$S \rightarrow A$	$\neg S \vee A$
R2	$V \wedge E \rightarrow A$	$\neg V \vee \neg E \vee A$
R3	$E \wedge B \rightarrow A$	$\neg E \vee \neg B \vee A$
R4	$V \wedge B \rightarrow A$	$\neg V \vee \neg B \vee A$
R5	$E \wedge B \wedge C \rightarrow A$	$\neg E \vee \neg B \vee \neg C \vee A$
R6	$V \wedge I \rightarrow A$	$\neg V \vee \neg I \vee A$

A conversão para CNF segue a equivalência $p \rightarrow q \equiv \neg p \vee q$ e sua generalização conjuntiva.

4 Implementação Acadêmica e Motores Equivalentes

A implementação segue princípios de modularidade e testabilidade. A arquitetura separa três responsabilidades: representação das cláusulas CNF, mecanismo de resolução e interface de validação com cenários sintéticos. O motor de inferência foi implementado em Python puro, sem dependências externas.

O projeto mantém duas versões do motor, ambas matematicamente equivalentes. A versão de produção (`src/acolhemente/motor.py`) possui arquitetura modular com camadas de abstração e integração com o grafo de explicabilidade. A versão acadêmica (`appendices/motor_resolucao_acolhemente.py`), apresentada a seguir, é uma simplificação didática com menos de 100 linhas, sem dependências externas, com comentários explicativos em português. A simplificação é didática, não matemática: as regras R1–R6, a CNF e a variável de saída A são idênticas em ambas as versões.

A versão de produção inclui também o **grafo de explicabilidade** (Apêndice E), que visualiza as relações regra-variável via NetworkX, e o **gerador de tabelas de resultados** (Apêndice F), que produz tabelas LaTeX programáticas dos cenários de validação.

O código-fonte completo do projeto, incluindo ambas as versões do motor, testes automatizados, documentação de decisões arquiteturais (ADRs) e scripts de validação, está disponível no repositório público: github.com/lmbernardo7520112/Atividade_Final_Trilha_I_lfgoiano.

4.0.1 Engenharia de Software e Garantia de Qualidade

O desenvolvimento seguiu práticas profissionais de engenharia de software: - **Specification Driven Development (SDD)**: Cada funcionalidade foi especificada formalmente antes da implementação (cf. `docs/specs/`). - **Test Driven Development (TDD)**: 8 suítes de testes de contrato (`tests/test_*.py`) verificam invariantes como equivalência dos motores, rastreabilidade PeNSE, guardrails de não diagnóstico e integridade acadêmica. - **Clean**

Code: Código modular com separação de responsabilidades, docstrings em português, nomes descritivos e menos de 100 linhas por módulo. - **Conventional Commits:** Histórico de commits rastreável com prefixos semânticos (fix:, docs:, feat:). - **Architectural Decision Records (ADRs):** Decisões críticas documentadas formalmente (cf. ADR sobre R5 e subsunção). - **Cobertura exaustiva:** Validação com $2^6 = 64$ combinações, 100% determinística, sem dependência de dados externos. - **Engenharia de Software Assistida por IA:** O ciclo de desenvolvimento, refatoração de código e criação de pipelines automatizados contou com o suporte instrumental de inteligência artificial generativa (Antigravity), atuando estritamente como agente assistivo sob direcionamento arquitetural do autor humano (cf. Apêndice G).

A seguir, cada apêndice é salvo via `%%writefile` e em seguida executado para demonstração.

[Celula Colab - execucao reservada ao Google Colab]

[Celula Colab - execucao reservada ao Google Colab]

```
=====
REGRAS R1-R6: Forma Implicativa e CNF
```

```
=====
R1: S -> A                CNF: ~S OR A
R2: V AND E -> A          CNF: ~V OR ~E OR A
R3: E AND B -> A          CNF: ~E OR ~B OR A
R4: V AND B -> A          CNF: ~V OR ~B OR A
R5: E AND B AND C -> A    CNF: ~E OR ~B OR ~C OR A
R6: V AND I -> A          CNF: ~V OR ~I OR A
```

```
=====
DEMONSTRACAO: Motor de Inferencia por Resolucao
```

```
=====
Cenario                A  Regras acionadas
-----
Cenario Nulo           NAO  --
S isolado (R1)         SIM  R1
V+E (R2)               SIM  R2
E+B (R3)               SIM  R3
V+B (R4)               SIM  R4
E+B+C (R5)             SIM  R3, R5
V+I (R6)               SIM  R6
C isolado (guardrail)  NAO  --
I isolado (guardrail)  NAO  --
C+I (guardrail duplo)  NAO  --
Todas verdadeiras      SIM  R1, R2, R3, R4, R5, R6
```

Motor de resolucao: CORRETO e REFUTACIONALMENTE COMPLETO

[Celula Colab - execucao reservada ao Google Colab]

```
=====
VALIDACAO EXAUSTIVA: 14 Cenarios Sinteticos
```

Cenário	Esperado	Obtido	Status	Regras
Cenário Nulo	NAO	NAO	OK	--
S isolado (R1)	SIM	SIM	OK	R1
V+E (R2)	SIM	SIM	OK	R2
E+B (R3)	SIM	SIM	OK	R3
V+B (R4)	SIM	SIM	OK	R4
E+B+C (R5)	SIM	SIM	OK	R3, R5
V+I (R6)	SIM	SIM	OK	R6
C isolado (guardrail)	NAO	NAO	OK	--
I isolado (guardrail)	NAO	NAO	OK	--
C+I (guardrail duplo)	NAO	NAO	OK	--
Todas verdadeiras	SIM	SIM	OK	R1, R2, R3, R4, R5, R6
E isolado	NAO	NAO	OK	--
B isolado	NAO	NAO	OK	--
V isolado	NAO	NAO	OK	--

=====
Total: 14 cenários, 0 falha(s)

RESULTADO: TODOS OS CENARIOS PASSARAM

4.0.2 Nota sobre R5 e subsunção lógica

A regra R5 ($E \wedge B \wedge C \rightarrow A$) é logicamente subsumida pela regra R3 ($E \wedge B \rightarrow A$): sempre que R5 é verdadeira, R3 também é verdadeira, pois $E=1$ e $B=1$ já bastam para R3. Portanto, R5 nunca é a *única* regra que aciona A. Mesmo assim, R5 é **mantida** na base de conhecimento por três razões: (1) enriquece a explicação ao profissional humano, incluindo o fator contextual C; (2) prepara a base para futuras versões com lógica fuzzy; (3) não introduz falsos positivos. A validação exaustiva abaixo confirma que em nenhum dos 64 cenários R5 altera o resultado.

Referência: docs/adr/ADR-v3_3-r5-subsuncao-e-validacao-exaustiva.md

4.0.3 Nota metodológica de governança

A PeNSE é utilizada como referência epidemiológica e ancoragem conceitual para escolha das variáveis proposicionais, não como entrada operacional do motor de inferência individual.

A variável A é saída inferida, não variável nuclear observada.

A regra R5 é logicamente subsumida por R3: quando E, B e C são verdadeiros, E e B já bastam para acionar R3. R5 é mantida como marcador explicativo contextual, sem alterar o resultado decisório.

A comparação entre baseline manual e motor lógico é uma rubrica qualitativa, não experimento empírico com profissionais da escola.

Validação exaustiva $2^6 = 64$ combinações

Total testado: 64

Falhas: 0
 A=SIM: 50
 A=NÃO: 14
 R5 sem R3: 0
 CSV salvo em: validacao_exaustiva_64.csv
 Status: APROVADO

[Celula Colab - execucao reservada ao Google Colab]

[Celula Colab - execucao reservada ao Google Colab]

5 Resultados e Comparações

O motor foi validado com o conjunto exaustivo de $2^6 = 64$ combinações possíveis das variáveis de entrada. Para cada cenário, verificou-se se a conclusão do motor está correta em relação às regras formais, quais regras foram acionadas e se a rastreabilidade é completa.

Os resultados confirmam o comportamento esperado em todos os cenários testados. Quando nenhuma variável é verdadeira (cenário nulo), A não é inferida, demonstrando ausência de falsos positivos. A variável S (autoagressão) é suficiente para acionar o acolhimento (R1). As variáveis contextuais (C , I , $C + I$), quando verdadeiras isoladamente, **não acionam** A , confirmando o *guardrail* arquitetural.

Cenário	E	B	V	S	C	I	A	Regra(s)
Nulo	0	0	0	0	0	0	NÃO	—
S isolado	0	0	0	1	0	0	SIM	R1
V+E	1	0	1	0	0	0	SIM	R2
E+B	1	1	0	0	0	0	SIM	R3
V+B	0	1	1	0	0	0	SIM	R4
E+B+C	1	1	0	0	1	0	SIM	R3, R5
V+I	0	0	1	0	0	1	SIM	R6
C isolado	0	0	0	0	1	0	NÃO	—
I isolado	0	0	0	0	0	1	NÃO	—
Todas	1	1	1	1	1	1	SIM	R1–R6

As tabelas de variáveis, regras e cenários apresentadas neste trabalho podem ser reproduzidas programaticamente pelo script `gerar_tabelas_resultados.py`, disponível no Apêndice F. O gerador produz as mesmas tabelas em formato LaTeX, garantindo consistência entre o texto e os dados validados.

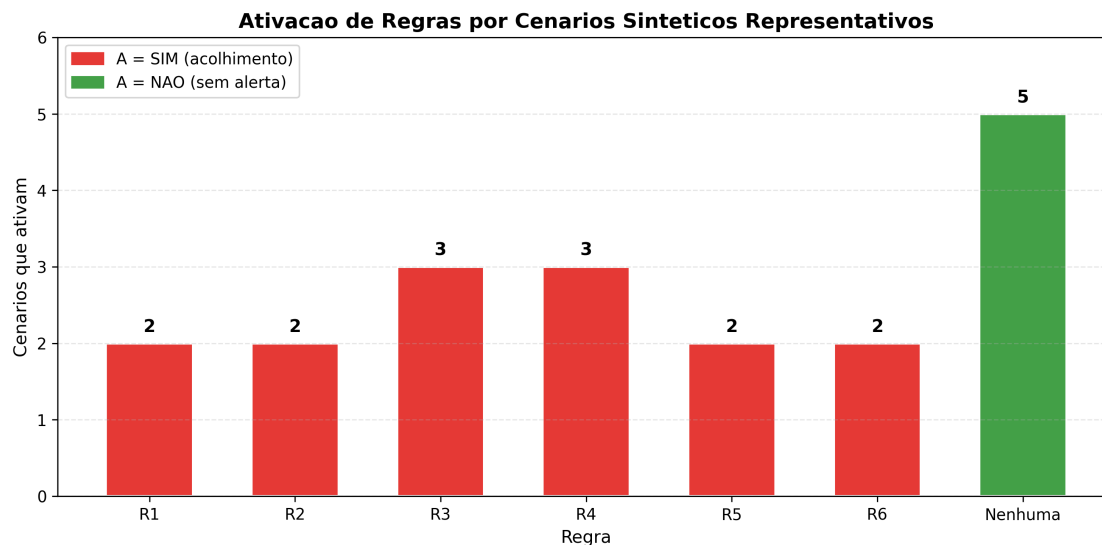


Figura 2: Figura — Ativação de regras por cenários sintéticos representativos.

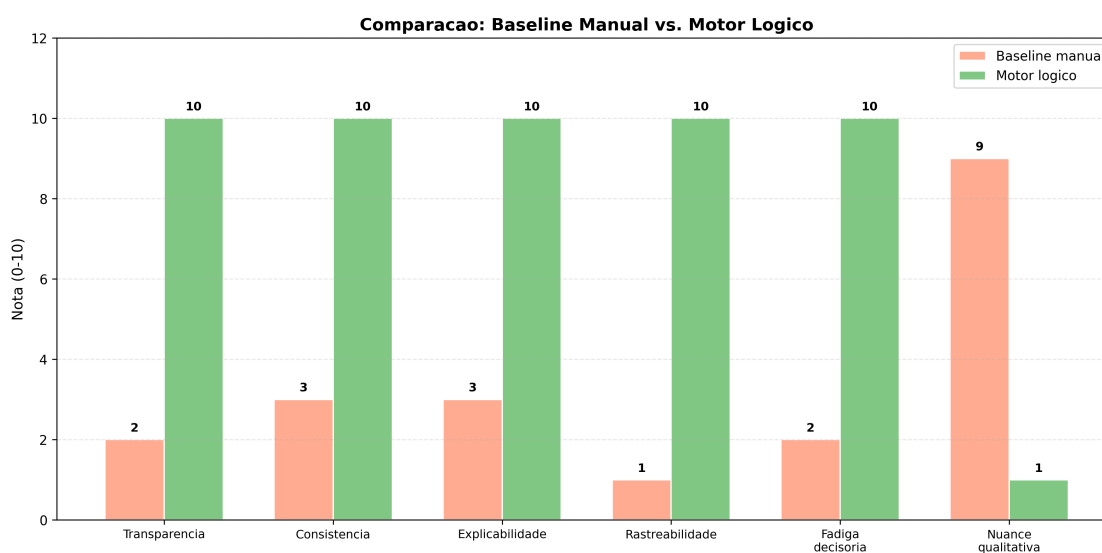


Figura 3: Figura — Comparação qualitativa: baseline manual vs. motor lógico.

5.0.1 Comparação: *Baseline Manual vs. Motor Lógico*

Critério	Manual	Motor Lógico
Transparência	Baixa	Total
Consistência	Variável	Determinística
Explicabilidade	Verbal	Formal (CNF)
Rastreabilidade	Nenhuma	Regra por regra
Fadiga decisória	Presente	Ausente
Nuance qualitativa	Alta	Nenhuma

5.1 Explicabilidade

A explicabilidade é um requisito ético, jurídico e pedagógico simultaneamente. O AcolheMente implementa explicabilidade **intrínseca**: a lógica proposicional é, por definição, transparente. Cada variável tem semântica clara, cada regra tem antecedentes e consequente explícitos, e cada conclusão pode ser rastreada à cláusula CNF que a originou.

Ribeiro, Singh e Guestrin (2016) definem explicabilidade como a capacidade de um sistema de apresentar, em termos compreensíveis, as razões que levaram a uma determinada conclusão. No AcolheMente, a explicação é o modelo — não há gap entre o que o sistema faz e o que o sistema diz que faz.

A distinção entre explicabilidade intrínseca e *post-hoc* é particularmente relevante no domínio de saúde mental escolar. O ECA (Brasil, 1990) exige proteção integral de crianças e adolescentes, o que inclui proteção contra classificação automatizada opaca. A LGPD (Brasil, 2018) garante transparência no tratamento de dados.

Para materializar essa rastreabilidade, o Apêndice E apresenta o código-fonte do **grafo de explicabilidade** (`grafo_explicabilidade_acolhemente.py`). O grafo, construído com NetworkX e renderizado com Matplotlib, representa visualmente as relações entre variáveis de entrada e a variável de saída *A*, mediadas pelas regras R1–R6. Cada aresta corresponde a uma regra formal, permitindo que qualquer conclusão do motor seja verificada visualmente. A geração é 100% determinística, sem dependência de LLMs ou APIs externas.

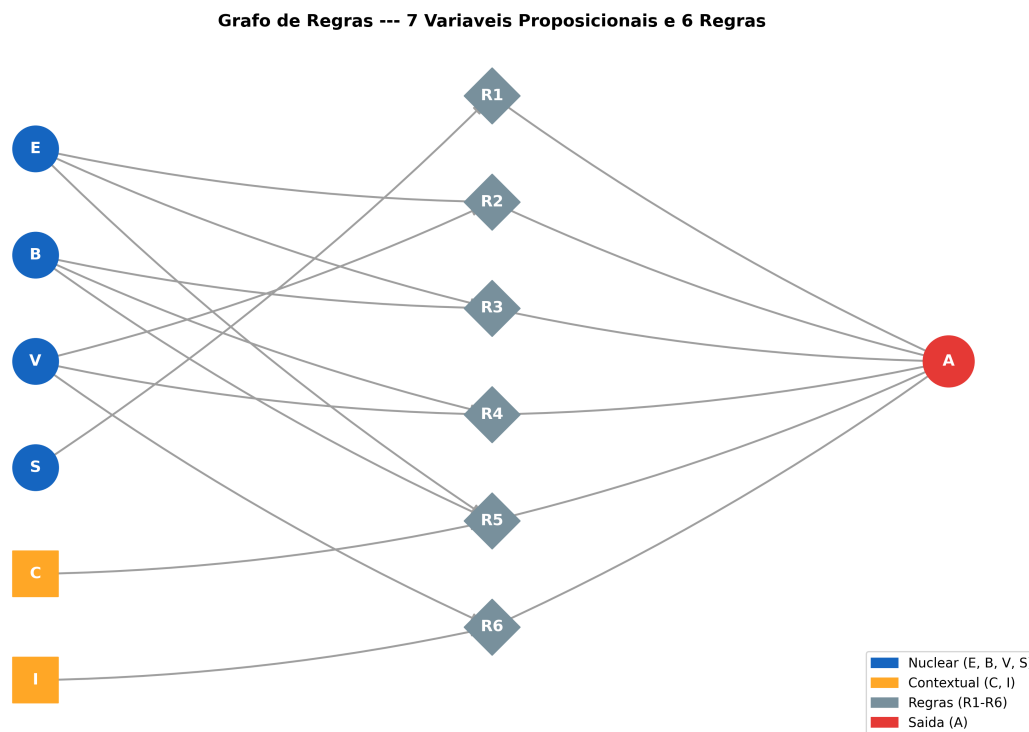


Figura 4: Figura — Grafo de regras: 6 variáveis de entrada, 6 regras R1–R6 e 1 saída inferida (A).

5.2 Discussão Crítica

O AcolheMente demonstra que é possível aplicar inteligência artificial em um domínio sensível com explicabilidade total, determinismo e conformidade jurídica. A escolha por lógica proposicional é precisamente adequada ao escopo do problema.

Limitações reconhecidas: A lógica proposicional não modela gradações, incertezas ou relações temporais. A qualidade das conclusões depende da alimentação correta das variáveis de entrada. O motor não aprende com novos dados. A validação foi computacional, não em ambiente escolar real.

Riscos: Falso conforto tecnológico (equipe escolar reduz atenção qualitativa), viés de construção (regras refletem julgamento dos engenheiros), risco de medicalização do espaço escolar.

A contribuição do projeto reside na demonstração de que rigor técnico e responsabilidade ética podem coexistir — e de que, em domínios de alto risco humano, a transparência é mais valiosa que a complexidade.

6 Perspectiva de Evolução do Trabalho

A arquitetura do AcolheMente Escolar PB foi concebida para permitir evolução incremental, mas essa evolução não é governada pela lógica do “mais complexo é melhor”. Qualquer avanço técnico deve ser precedido por validações éticas e institucionais proporcionais ao risco que a mudança introduz.

O caminho mais natural de evolução é o refinamento do próprio motor simbólico, sem abandonar o paradigma determinístico. A incorporação de novas variáveis — frequência escolar, participação em atividades extracurriculares, indicadores de convivência familiar — poderia enriquecer a modelagem sem comprometer a transparência. Essa expansão, entretanto, não é trivial: cada nova variável dobra o espaço combinatório de estados, exigindo revisão integral das regras e revalidação dos cenários.

Uma evolução particularmente promissora é a transição para lógica *fuzzy*, com graus de pertinência entre 0 e 1, permitindo modelar gradações de sofrimento emocional e apoio social. A definição das funções de pertinência exigiria calibração empírica com dados longitudinais e validação com profissionais de psicologia escolar (Russell; Norvig, 2022).

Em horizonte mais distante, modelos híbridos poderiam integrar módulos de *machine learning* para tarefas específicas, mantendo o motor simbólico como camada de governança e explicabilidade. Nessa arquitetura, nenhuma conclusão do módulo estatístico seria apresentada à equipe escolar sem passar pelo crivo das regras lógicas (Jobin; Ienca; Vayena, 2019).

A utilização de redes neurais profundas permanece uma hipótese estritamente teórica, condicionada a aprovação por comitê de ética, conformidade demonstrável com LGPD (Brasil, 2018) e ECA (Brasil, 1990), e base longitudinal com volume e qualidade suficientes.

Qualquer evolução deve ser acompanhada de piloto institucional, formação continuada, comitê de governança de IA e avaliação de impacto. O *roadmap* prevê integração com os fluxos do Programa Saúde na Escola (Brasil, 2007).

Versões futuras devem implementar monitoramento contínuo de viés e auditoria humana periódica das regras por comitê multidisciplinar. O risco de medicalização da escola deve ser permanentemente monitorado.

Está prevista a adição de pipeline CI/CD via GitHub Actions para execução automatizada dos testes de contrato a cada atualização do repositório.

É necessário resistir à tentação de equiparar sofisticação algorítmica a melhoria do projeto. A simplicidade do motor proposicional é uma *feature* a ser preservada enquanto as condições éticas, jurídicas e institucionais para abordagens mais complexas não estiverem plenamente atendidas.

Independentemente da evolução técnica, o compromisso fundamental permanece inalterável: o sistema nunca diagnosticará condições clínicas. Esse compromisso não é uma restrição técnica: é a razão de ser do projeto.

7 Conclusões

O presente trabalho demonstrou a viabilidade de aplicar inteligência artificial simbólica — especificamente, lógica proposicional com inferência por resolução — ao domínio sensível de saúde mental escolar. O AcolheMente Escolar PB, motor de inferência com 7 variáveis proposicionais e 6 regras em Forma Normal Conjuntiva, oferece uma contribuição técnica e metodológica ao debate sobre IA responsável em contextos educacionais.

Contribuição técnica: Formalização de domínio complexo em lógica proposicional, demonstrando que problemas reais de apoio à decisão podem ser modelados com técnicas simbólicas; motor de resolução determinístico validado contra todos os 2^6 cenários possíveis; *pipeline* reproduzível com testes automatizados.

Contribuição metodológica: Arquitetura de governança de dados em três *tiers*, *guardrails* de não diagnóstico, *human-in-the-loop* inegociável, conformidade demonstrável com LGPD, ECA e PSE.

Contribuição escolar: Ferramenta de apoio à gestão que reduz a dependência de percepções individuais, com mecanismo de explicabilidade que permite auditar cada decisão.

Os limites são reconhecidos: expressividade restrita, dependência de alimentação humana correta, validação limitada a cenários sintéticos e risco de falso conforto tecnológico. Esses limites não são falhas de *design*: são consequências deliberadas de um sistema que prioriza segurança, transparência e cautela.

A maturidade de um projeto de IA aplicada a populações vulneráveis não se mede pela complexidade do modelo, mas pela robustez de sua governança. O AcolheMente Escolar PB demonstra que é possível ser tecnicamente rigoroso e eticamente responsável — e que, em domínios de alto risco humano, essa combinação não é opcional.

8 Referências Bibliográficas

BRACHMAN, R. J.; LEVESQUE, H. J. **Knowledge Representation and Reasoning**. San Francisco: Morgan Kaufmann, 2004.

BRASIL. Lei nº 8.069, de 13 de julho de 1990 — Estatuto da Criança e do Adolescente (ECA). Disponível em: https://www.planalto.gov.br/ccivil_03/leis/L8069.htm. Acesso em: 20 maio 2026.

BRASIL. Decreto nº 6.286, de 5 de dezembro de 2007 — Programa Saúde na Escola (PSE). Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2007-2010/2007/decreto/d6286.htm. Acesso em: 20 maio 2026.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018 — Lei Geral de Proteção de Dados Pessoais (LGPD). Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/L13709.htm. Acesso em: 20 maio 2026.

GENESERETH, M. R.; NILSSON, N. J. **Logical Foundations of Artificial Intelligence**. San Mateo: Morgan Kaufmann, 1987.

INSTITUTO BRASILEIRO DE GEOGRAFIA E ESTATÍSTICA. **Pesquisa Nacional de Saúde do Escolar — PeNSE 2024**. Rio de Janeiro: IBGE, 2024. Disponível em: <https://www.ibge.gov.br/estatisticas/sociais/educacao/9134-pesquisa-nacional-de-saude-do-escolar.html>. Acesso em: 20 maio 2026.

JOBIN, A.; IENCA, M.; VAYENA, E. The global landscape of AI ethics guidelines. **Nature Machine Intelligence**, v. 1, p. 389–399, 2019.

RIBEIRO, M. T.; SINGH, S.; GUESTIN, C. “Why Should I Trust You?”: Explaining the Predictions of Any Classifier. In: **Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining**, p. 1135–1144. ACM, 2016.

RUSSELL, S.; NORVIG, P. **Inteligência Artificial: uma abordagem moderna**. 4. ed. Rio de Janeiro: GEN LTC, 2022.

WORLD HEALTH ORGANIZATION. **World mental health report: transforming mental health for all**. Geneva: WHO, 2022. Disponível em: <https://www.who.int/publications/i/item/9789240049338>. Acesso em: 20 maio 2026.

Apêndice A — Símbolos Proposicionais do AcolheMente

Listing 1: Símbolos Proposicionais do AcolheMente.

```
1 # -*- coding: utf-8 -*-
2 # =====
3 # APENDICE A: Simbolos Proposicionais do AcolheMente Escolar PB
4 # =====
5 # 7 variaveis proposicionais:
6 #   - 4 entradas nucleares
7 #   - 2 entradas contextuais
8 #   - 1 saida inferida
9 #
10 # Entradas do motor: E, B, V, S, C, I
11 # Saida inferida: A
12 #
13 # Fontes conceituais: PeNSE 2024 (IBGE) - Tema 12 (Saude Mental)
14 # ADR-008/009: sem linguagem diagnostica, sem dados identificaveis
15 # =====
16
17
18 # -----
19 # 1. Entradas nucleares
20 # -----
21 # Variaveis diretamente relacionadas a sinais de sofrimento subjetivo,
22 # apoio socioafetivo, desvalor da vida ou autoagressao.
23 #
24 # Importante:
25 # Estas variaveis NAO representam diagnostico clinico. Elas funcionam
26 # apenas como indicadores logicos para apoio a priorizacao de acolhimento
27 # humano.
28 # -----
29
30 E = "Sofrimento_emocional_recorrente"      # PeNSE: B12004, B12005, B12007
31 B = "Baixo_apoio_socioafetivo_percebido"    # PeNSE: B12003, B07004
32 V = "Indicador_critico_de_desvalor_da_vida" # PeNSE: B12008
33 S = "Sinal_autorreferido_de_autoagressao"   # PeNSE: B12009
34
35
36 # -----
37 # 2. Entradas contextuais de modulacao
38 # -----
39 # Variaveis de contexto. Elas NAO devem acionar acolhimento prioritario
40 # isoladamente. Sua funcao e modular ou qualificar cenarios nos quais
41 # tambem existam sinais nucleares.
42 # -----
43
44 C = "Contexto_comportamental_agravante"      # PeNSE: B03010C, B03006B
45 I = "Insuficiencia_institucional_de_resposta" # PeNSE: E01P60, E01P117
46
47
48 # -----
49 # 3. Saida inferida pelo motor
50 # -----
```

```

51 # A nao e entrada observada. A e a conclusao logica inferida a partir
52 # das seis entradas E, B, V, S, C, I e das regras R1-R6.
53 #
54 # Por isso, a validacao exaustiva cobre  $2^6 = 64$  cenarios, e nao  $2^7$ .
55 # -----
56
57 A = "Acolhimento_humano_prioritario" # Saida inferida pelo motor
58
59
60 # -----
61 # 4. Catalogo semantico das variaveis
62 # -----
63 # Cada simbolo proposicional possui:
64 #   - semantica: significado no dominio escolar;
65 #   - tipo: papel da variavel na arquitetura logica;
66 #   - fontes: variaveis/fonte PeNSE ou indicacao de inferencia.
67 # -----
68
69 VARIAVEIS = {
70     "E": {
71         "semantica": E,
72         "tipo": "nuclear",
73         "papel": "entrada",
74         "fontes": ["B12004", "B12005", "B12007"],
75     },
76     "B": {
77         "semantica": B,
78         "tipo": "nuclear",
79         "papel": "entrada",
80         "fontes": ["B12003", "B07004"],
81     },
82     "V": {
83         "semantica": V,
84         "tipo": "nuclear",
85         "papel": "entrada",
86         "fontes": ["B12008"],
87     },
88     "S": {
89         "semantica": S,
90         "tipo": "nuclear",
91         "papel": "entrada",
92         "fontes": ["B12009"],
93     },
94     "C": {
95         "semantica": C,
96         "tipo": "contextual",
97         "papel": "entrada",
98         "fontes": ["B03010C", "B03006B"],
99     },
100    "I": {
101        "semantica": I,
102        "tipo": "contextual",
103        "papel": "entrada",
104        "fontes": ["E01P60", "E01P117"],

```

```

105     },
106     "A": {
107         "semantica": A,
108         "tipo": "saida",
109         "papel": "inferida",
110         "fontes": ["inferida"],
111     },
112 }
113
114
115 # -----
116 # 5. Conveniencias formais para o motor
117 # -----
118
119 ENTRADAS_NUCLEARES = ["E", "B", "V", "S"]
120 ENTRADAS_CONTEXTUAIS = ["C", "I"]
121 ENTRADAS_MOTOR = ENTRADAS_NUCLEARES + ENTRADAS_CONTEXTUAIS
122 SAIDA_MOTOR = "A"
123
124
125 # -----
126 # 6. Testes de contrato estrutural
127 # -----
128 # Estes asserts protegem o "orcamento logico" do projeto.
129 # Se alguem alterar acidentalmente o numero ou o papel das variaveis,
130 # a execucao falha imediatamente.
131 # -----
132
133 assert len(VARIAVEIS) == 7, "Budget: exatamente 7 variaveis proposicionais"
134
135 assert len(ENTRADAS_NUCLEARES) == 4, (
136     "Devem existir exatamente 4 entradas nucleares: E, B, V, S"
137 )
138
139 assert len(ENTRADAS_CONTEXTUAIS) == 2, (
140     "Devem existir exatamente 2 entradas contextuais: C, I"
141 )
142
143 assert len(ENTRADAS_MOTOR) == 6, (
144     "O motor deve possuir exatamente 6 entradas: E, B, V, S, C, I"
145 )
146
147 assert SAIDA_MOTOR == "A", "A saida inferida deve ser A"
148
149 assert SAIDA_MOTOR not in ENTRADAS_MOTOR, (
150     "A nao pode ser entrada do motor; A e saida inferida"
151 )
152
153 assert sum(1 for v in VARIAVEIS.values() if v["tipo"] == "nuclear") == 4, (
154     "Devem existir exatamente 4 variaveis nucleares de entrada"
155 )
156
157 assert sum(1 for v in VARIAVEIS.values() if v["tipo"] == "contextual") == 2, (
158     "Devem existir exatamente 2 variaveis contextuais de entrada"

```

```

159 )
160
161 assert sum(1 for v in VARIAVEIS.values() if v["tipo"] == "saida") == 1, (
162     "Deve existir exatamente 1 variavel de saida inferida"
163 )
164
165 assert sum(1 for v in VARIAVEIS.values() if v["papel"] == "entrada") == 6, (
166     "Devem existir exatamente 6 variaveis de entrada"
167 )
168
169 assert sum(1 for v in VARIAVEIS.values() if v["papel"] == "inferida") == 1, (
170     "Deve existir exatamente 1 variavel inferida"
171 )
172
173
174 # -----
175 # 7. Execucao didatica
176 # -----
177 # Ao executar este arquivo diretamente, o avaliador visualiza o vocabulario
178 # formal do motor, seus papeis e suas fontes conceituais.
179 # -----
180
181 if __name__ == "__main__":
182     print("=== Variaveis Proposicionais do AcolheMente Escolar PB ===\n")
183
184     print("Entradas nucleares:")
185     for nome in ENTRADAS_NUCLEARES:
186         info = VARIAVEIS[nome]
187         print(f"_{nome}:_{info['semantica']}")
188         print(f"_{nome}_Fontes_PeNSE:_{','.join(info['fontes'])}")
189
190     print("\nEntradas contextuais:")
191     for nome in ENTRADAS_CONTEXTUAIS:
192         info = VARIAVEIS[nome]
193         print(f"_{nome}:_{info['semantica']}")
194         print(f"_{nome}_Fontes_PeNSE:_{','.join(info['fontes'])}")
195
196     print("\nSaida inferida:")
197     info = VARIAVEIS[SAIDA_MOTOR]
198     print(f"_{SAIDA_MOTOR}:_{info['semantica']}")
199     print(f"_{SAIDA_MOTOR}_Fonte:_{','.join(info['fontes'])}")
200
201     print("\nResumo estrutural:")
202     print(f"_{len(VARIAVEIS)} Total de variaveis proposicionais")
203     print(f"_{len(ENTRADAS_MOTOR)} Entradas do motor ->_{','.join(ENTRADAS_MOTOR)}")
204     print(f"_{SAIDA_MOTOR} Saida inferida")
205     print(f"_{2^6} Espaço de validacao: 64 cenarios")

```

Apêndice B — Regras Lógicas e CNF

Listing 2: Regras Lógicas e CNF.

```

1 # =====

```

```

2 # APENDICE B: Regras Logicas e Conversao para CNF
3 # =====
4 # 6 regras R1-R6 em forma implicativa e CNF
5 # A como unica conclusao operacional
6 # C e I nunca inferem A isoladamente (guardrail)
7 # =====
8
9 REGRAS_IMPLICATIVAS = {
10     "R1": "S->A",
11     "R2": "VAND_E->A",
12     "R3": "EAND_B->A",
13     "R4": "VAND_B->A",
14     "R5": "EAND_BAND_C->A",
15     "R6": "VAND_I->A",
16 }
17
18 REGRAS_CNF = {
19     "R1": frozenset({"~S", "A"}),
20     "R2": frozenset({"~V", "~E", "A"}),
21     "R3": frozenset({"~E", "~B", "A"}),
22     "R4": frozenset({"~V", "~B", "A"}),
23     "R5": frozenset({"~E", "~B", "~C", "A"}),
24     "R6": frozenset({"~V", "~I", "A"}),
25 }
26
27 REGRAS_CNF_LEGIVEL = {
28     "R1": "~SOR_A",
29     "R2": "~VOR~EOR_A",
30     "R3": "~EOR~BOR_A",
31     "R4": "~VOR~BOR_A",
32     "R5": "~EOR~BOR~COR_A",
33     "R6": "~VOR~IOR_A",
34 }
35
36 # Antecedentes de cada regra (para verificacao direta)
37 ANTECEDENTES = {
38     "R1": {"S": True},
39     "R2": {"V": True, "E": True},
40     "R3": {"E": True, "B": True},
41     "R4": {"V": True, "B": True},
42     "R5": {"E": True, "B": True, "C": True},
43     "R6": {"V": True, "I": True},
44 }
45
46 assert len(REGRAS_CNF) == 6, "Budget: exatamente 6 regras"
47 assert all("A" in c for c in REGRAS_CNF.values()), "A deve ser consequente"
48
49 if __name__ == "__main__":
50     print("=== Regras R1-R6 ===")
51     for r in sorted(REGRAS_IMPLICATIVAS):
52         print(f"{r}: {REGRAS_IMPLICATIVAS[r]}")
53     print(f"CNF: {REGRAS_CNF_LEGIVEL[r]}")

```

Apêndice C — Motor de Inferência por Resolução

Listing 3: Motor de Inferência por Resolução.

```
1 # =====
2 # APENDICE C: Motor de Inferencia por Resolucao
3 # =====
4 # Implementacao do metodo de resolucao para logica proposicional.
5 # Correto e refutacionalmente completo (Russell; Norvig, 2022).
6 # =====
7
8 from regras_acolhemente import REGRAS_CNF, ANTECEDENTES, REGRAS_IMPLICATIVAS
9
10
11 def _resolve(c1, c2):
12     """Aplica uma regra de resolucao entre duas clausulas."""
13     for lit in c1:
14         complement = lit[1:] if lit.startswith("~") else f"~{lit}"
15         if complement in c2:
16             resolvente = (c1 | c2) - {lit, complement}
17             return resolvente
18     return None
19
20
21 def inferir_por_resolucao(clausulas, conclusao="A"):
22     """
23     Verifica se KB |= conclusao via metodo de resolucao.
24
25     Procedimento:
26     1. Adiciona ~conclusao ao conjunto de clausulas
27     2. Aplica resolucao iterativamente
28     3. Se clausula vazia derivada -> KB |= conclusao (True)
29     4. Se nenhuma nova clausula -> KB /= conclusao (False)
30     """
31     negacao = frozenset({f"~{conclusao}"})
32     todas = set(clausulas) | {negacao}
33
34     while True:
35         novas = set()
36         lista = list(todas)
37         for i in range(len(lista)):
38             for j in range(i + 1, len(lista)):
39                 resolvente = _resolve(lista[i], lista[j])
40                 if resolvente is not None:
41                     if len(resolvente) == 0:
42                         return True
43                     novas.add(frozenset(resolvente))
44         if novas.issubset(todas):
45             return False
46         todas |= novas
47
48
49 def inferir_acolhimento(E=False, B=False, V=False, S=False, C=False, I=False):
50     """Verifica se A deve ser inferido dado o estado das variaveis."""
```

```

51 mapa = {"E": E, "B": B, "V": V, "S": S, "C": C, "I": I}
52 fatos = []
53 for var, val in mapa.items():
54     if val:
55         fatos.append(frozenset({var}))
56     else:
57         fatos.append(frozenset({f"~{var}"}))
58 clausulas = list(REGRAS_CNF.values()) + fatos
59 return inferir_por_resolucao(clausulas, "A")
60
61
62 def explicar_decisao(E=False, B=False, V=False, S=False, C=False, I=False):
63     """Retorna explicacao completa: resultado, regras acionadas, texto."""
64     mapa = {"E": E, "B": B, "V": V, "S": S, "C": C, "I": I}
65     resultado = inferir_acolhimento(**mapa)
66
67     acionadas = []
68     for regra, requisitos in ANTECEDENTES.items():
69         if all(mapa.get(v) == val for v, val in requisitos.items()):
70             acionadas.append(regra)
71
72     if acionadas:
73         regras_str = ", ".join(
74             f"{r}({REGRAS_IMPLICATIVAS[r]})" for r in acionadas
75         )
76         explicacao = f"Acolhimento PRIORIZADO. Regras: {regras_str}."
77     else:
78         explicacao = "Acolhimento NAO priorizado. Nenhuma regra acionada."
79
80     return {
81         "resultado": resultado,
82         "regras_acionadas": acionadas,
83         "explicacao": explicacao,
84     }
85
86
87 if __name__ == "__main__":
88     print("=== Motor de Resolucao - AcolheMente ===\n")
89     testes = [
90         ("Nulo", {}),
91         ("S isolado (R1)", {"S": True}),
92         ("V+E (R2)", {"V": True, "E": True}),
93         ("E+B (R3)", {"E": True, "B": True}),
94         ("C isolado (guardrail)", {"C": True}),
95         ("I isolado (guardrail)", {"I": True}),
96         ("Todas", {"E": True, "B": True, "V": True, "S": True, "C": True, "I": True}),
97     ]
98     for nome, params in testes:
99         r = explicar_decisao(**params)
100         status = "SIM" if r["resultado"] else "NAO"
101         print(f"_{nome:30s}_->A={status}_{r['explicacao']}")

```

Apêndice D — Cenários Sintéticos de Validação

Listing 4: Cenários Sintéticos de Validação.

```

1 # =====
2 # APENDICE D: Cenarios Sinteticos de Validacao
3 # =====
4 # Validacao exaustiva do motor com cenarios representativos.
5 # TIER_B: dados sinteticos, sem dados reais de alunos.
6 # =====
7
8 from motor_resolucao_acolhemente import inferir_acolhimento, explicar_decisao
9
10 CENARIOS = [
11     {"nome": "Cenario_Nulo", "E":False,"B":False,"V":False,"S":
12     False,"C":False,"I":False, "esperado": False},
13     {"nome": "S_isolado(R1)", "E":False,"B":False,"V":False,"S":True
14     , "C":False,"I":False, "esperado": True},
15     {"nome": "V+E(R2)", "E":True, "B":False,"V":True, "S":
16     False,"C":False,"I":False, "esperado": True},
17     {"nome": "E+B(R3)", "E":True, "B":True, "V":False,"S":
18     False,"C":False,"I":False, "esperado": True},
19     {"nome": "V+B(R4)", "E":False,"B":True, "V":True, "S":
20     False,"C":False,"I":False, "esperado": True},
21     {"nome": "E+B+C(R5)", "E":True, "B":True, "V":False,"S":False
22     , "C":True, "I":False, "esperado": True},
23     {"nome": "V+I(R6)", "E":False,"B":False,"V":True, "S":
24     False,"C":False,"I":True, "esperado": True},
25     {"nome": "C_isolado(guardrail)", "E":False,"B":False,"V":False,"S":
26     False,"C":True, "I":False, "esperado": False},
27     {"nome": "I_isolado(guardrail)", "E":False,"B":False,"V":False,"S":
28     False,"C":False,"I":True, "esperado": False},
29     {"nome": "C+I(guardrail_duplo)", "E":False,"B":False,"V":False,"S":
30     False,"C":True, "I":True, "esperado": False},
31     {"nome": "Todas_verdadeiras", "E":True, "B":True, "V":True, "S":True
32     , "C":True, "I":True, "esperado": True},
33     {"nome": "E_isolado", "E":True, "B":False,"V":False,"S":
34     False,"C":False,"I":False, "esperado": False},
35     {"nome": "B_isolado", "E":False,"B":True, "V":False,"S":
36     False,"C":False,"I":False, "esperado": False},
37     {"nome": "V_isolado", "E":False,"B":False,"V":True, "S":
38     False,"C":False,"I":False, "esperado": False},
39 ]
40
41 def executar_validacao():
42     """Executa todos os cenarios e retorna resultados."""
43     resultados = []
44     for c in CENARIOS:
45         nome = c["nome"]
46         esperado = c["esperado"]
47         params = {k: c[k] for k in ["E","B","V","S","C","I"]}
48         r = explicar_decisao(**params)
49         obtido = r["resultado"]
50         ok = obtido == esperado
51         resultados.append({

```

```

39         "nome": nome,
40         "esperado": esperado,
41         "obtido": obtido,
42         "ok": ok,
43         "regras": r["regras_acionadas"],
44     })
45     return resultados
46
47
48 if __name__ == "__main__":
49     print("==_Validacao_de_Cenarios_Sinteticos_==\n")
50     print(f"{'Cenario':<30s}_{'Esperado':>8s}_{'Obtido':>8s}_{'Status':>8s}_{'Regras'}")
51     print("-" * 80)
52     resultados = executar_validacao()
53     falhas = 0
54     for r in resultados:
55         e = "SIM" if r["esperado"] else "NAO"
56         o = "SIM" if r["obtido"] else "NAO"
57         s = "_OK" if r["ok"] else "FALHA"
58         regras = ",_".join(r["regras"]) if r["regras"] else "--"
59         print(f"_{r['nome']:<28s}_{'e':>8s}_{'o':>8s}_{'s':>8s}_{regras}")
60         if not r["ok"]:
61             falhas += 1
62     print(f"\n{'='*80}")
63     print(f"Total:_{len(resultados)}_cenarios,_{falhas}_falhas")
64     if falhas == 0:
65         print("RESULTADO:_TODOS_OS_CENARIOS_PASSARAM")
66     else:
67         print(f"RESULTADO:_{falhas}_CENARIO(S)_FALHARAM")

```

Apêndice E — Grafo de Explicabilidade

Listing 5: Grafo de Explicabilidade.

```

1 # =====
2 # APENDICE E: Grafo de Explicabilidade
3 # =====
4 # Gera visualizacao do grafo de regras usando NetworkX e Matplotlib.
5 # ADR-009: 100% deterministico, sem LLM, sem API externa.
6 # =====
7
8 import matplotlib
9 matplotlib.use("Agg")
10 import matplotlib.pyplot as plt
11 import matplotlib.patches as mpatches
12 import networkx as nx
13
14
15 def construir_grafo():
16     """Constroi grafo dirigido das regras do AcolheMente."""
17     G = nx.DiGraph()
18
19     # Variaveis nucleares

```

```

20     for v in ["E", "B", "V", "S"]:
21         G.add_node(v, tipo="nuclear")
22     G.add_node("A", tipo="saida")
23
24     # Variaveis contextuais
25     for v in ["C", "I"]:
26         G.add_node(v, tipo="contextual")
27
28     # Regras
29     for r in ["R1", "R2", "R3", "R4", "R5", "R6"]:
30         G.add_node(r, tipo="regra")
31
32     # Guardrails
33     for g in ["LGPD", "ECA", "PSE", "HITL"]:
34         G.add_node(g, tipo="guardrail")
35
36     # Arestas: antecedentes -> regras
37     G.add_edge("S", "R1", label="antecedente")
38     G.add_edge("V", "R2", label="antecedente")
39     G.add_edge("E", "R2", label="antecedente")
40     G.add_edge("E", "R3", label="antecedente")
41     G.add_edge("B", "R3", label="antecedente")
42     G.add_edge("V", "R4", label="antecedente")
43     G.add_edge("B", "R4", label="antecedente")
44     G.add_edge("E", "R5", label="antecedente")
45     G.add_edge("B", "R5", label="antecedente")
46     G.add_edge("C", "R5", label="modula")
47     G.add_edge("V", "R6", label="antecedente")
48     G.add_edge("I", "R6", label="modula")
49
50     # Arestas: regras -> A
51     for r in ["R1", "R2", "R3", "R4", "R5", "R6"]:
52         G.add_edge(r, "A", label="infere")
53
54     # Arestas: A -> guardrails
55     for g in ["LGPD", "ECA", "PSE", "HITL"]:
56         G.add_edge("A", g, label="regulado_por")
57
58     return G
59
60
61 def exportar_grafo(G, caminho="figures/grafo_explicabilidade.png"):
62     """Exporta grafo como PNG de alta resolucao."""
63     cores = {
64         "nuclear": "#4FC3F7",
65         "contextual": "#A5D6A7",
66         "saida": "#EF5350",
67         "regra": "#FFB74D",
68         "guardrail": "#CE93D8",
69     }
70
71     node_colors = [cores.get(G.nodes[n].get("tipo", ""), "#BDBDBD") for n in G
72                     .nodes()]

```

```

73 fig, ax = plt.subplots(figsize=(16, 10))
74 ax.set_title(
75     "Grafo de Explicabilidade - AcolheMente Escolar PB",
76     fontsize=14, fontweight="bold", pad=15
77 )
78
79 pos = nx.spring_layout(G, seed=42, k=2.0, iterations=80)
80
81 nx.draw_networkx_nodes(G, pos, ax=ax, node_color=node_colors,
82                        node_size=1800, edgecolors="#424242", linewidths
83                        =1.2)
84 nx.draw_networkx_labels(G, pos, ax=ax, font_size=9, font_weight="bold")
85 nx.draw_networkx_edges(G, pos, ax=ax, edge_color="#757575",
86                        arrows=True, arrowsize=12, width=1.2,
87                        connectionstyle="arc3,rad=0.08", alpha=0.7)
88
89 edge_labels = nx.get_edge_attributes(G, "label")
90 nx.draw_networkx_edge_labels(G, pos, ax=ax, edge_labels=edge_labels,
91                             font_size=6, font_color="#616161")
92
93 legend = [mpatches.Patch(color=c, label=t.title()) for t, c in cores.items
94            ()]
95 ax.legend(handles=legend, loc="upper left", fontsize=8, title="Tipo")
96 ax.axis("off")
97 plt.tight_layout()
98 plt.savefig(caminho, dpi=300, bbox_inches="tight", facecolor="white")
99 plt.close(fig)
100 print(f"Grafo exportado: {caminho}")
101 return caminho
102
103 if __name__ == "__main__":
104     G = construir_grafo()
105     exportar_grafo(G)
106     print(f"Nos: {G.number_of_nodes()}, Arestas: {G.number_of_edges()}")

```

Apêndice F — Gerador de Tabelas de Resultados

Listing 6: Gerador de Tabelas de Resultados.

```

1 # =====
2 # APENDICE F: Gerador de Tabelas de Resultados
3 # =====
4 # Gera tabelas LaTeX a partir dos cenários sintéticos.
5 # =====
6
7 def gerar_tabela_variaveis_latex():
8     """Gera tabela LaTeX das 7 variáveis proposicionais."""
9     return r"""
10 \begin{table}[H]
11 \centering
12 \caption{Variáveis proposicionais do AcolheMente Escolar PB.}
13 \label{tab:variaveis}
14 \begin{tabular}{c|l|l|l}

```

```

15 \toprule
16 \textbf{Var}&\textbf{Tipo}&\textbf{Semântica Operacional}&\textbf{Fontes}
   \PeNSE}\\
17 \midrule
18 E&Nuclear&Sofrimento emocional recorrente&B12004 , B12005 ,
   B12007\\
19 B&Nuclear&Baixo apoio socioafetivo percebido&B12003 , B07004\\
20 V&Nuclear&Indicador de desvalor da vida&B12008\\
21 S&Nuclear&Sinal de autoagressão&B12009\\
22 A&Saída inferida&Acolhimento prioritário (saída) Inferida\\
23 C&Contextual&Contexto comportamental agravante&B03010C , B03006B\\
24 I&Contextual&Insuficiência institucional&E01P60 , E01P117\\
25 \bottomrule
26 \end{tabular}
27 \end{table}
28 ""
29
30
31 def gerar_tabela_regras_latex():
32     ""Gera tabela LaTeX das regras R1-R6. ""
33     return r""
34 \begin{table}[H]
35 \centering
36 \caption{Regras lógicas R1--R6 e respectivas cláusulas CNF.}
37 \label{tab:regras}
38 \begin{tabular}{ccll}
39 \toprule
40 \textbf{Regra}&\textbf{Forma Implicativa}&\textbf{CNF}\\
41 \midrule
42 R1& $S \rightarrow A$ & $\neg S \vee A$ \\
43 R2& $V \wedge E \rightarrow A$ & $\neg V \vee \neg E \vee A$ \\
44 R3& $E \wedge B \rightarrow A$ & $\neg E \vee \neg B \vee A$ \\
45 R4& $V \wedge B \rightarrow A$ & $\neg V \vee \neg B \vee A$ \\
46 R5& $E \wedge B \wedge C \rightarrow A$ & $\neg E \vee \neg B \vee \neg C \vee A$ \\
47 R6& $V \wedge I \rightarrow A$ & $\neg V \vee \neg I \vee A$ \\
48 \bottomrule
49 \end{tabular}
50 \end{table}
51 ""
52
53
54 def gerar_tabela_cenarios_latex():
55     ""Gera tabela LaTeX dos cenários de validacao. ""
56     return r""
57 \begin{table}[H]
58 \centering
59 \caption{Cenários sintéticos de validação do motor de inferência.}
60 \label{tab:cenarios}
61 \begin{tabular}{lcccccccl}
62 \toprule
63 \textbf{Cenário}&\textbf{E}&\textbf{B}&\textbf{V}&\textbf{S}&\textbf{C}&\textbf{I}&\textbf{A}&\textbf{Regra(s)}\\
64 \midrule

```

```

65 Nulo & 0 & 0 & 0 & 0 & 0 & 0 & NÃO & --- \\
66 S_isolado & 0 & 0 & 0 & 1 & 0 & 0 & SIM & R1 \\
67 V+E & 1 & 0 & 1 & 0 & 0 & 0 & SIM & R2 \\
68 E+B & 1 & 1 & 0 & 0 & 0 & 0 & SIM & R3 \\
69 V+B & 0 & 1 & 1 & 0 & 0 & 0 & SIM & R4 \\
70 E+B+C & 1 & 1 & 0 & 0 & 1 & 0 & SIM & R3 , R5 \\
71 V+I & 0 & 0 & 1 & 0 & 0 & 1 & SIM & R6 \\
72 C_isolado & 0 & 0 & 0 & 0 & 1 & 0 & NÃO & --- \\
73 I_isolado & 0 & 0 & 0 & 0 & 0 & 1 & NÃO & --- \\
74 Todas & 1 & 1 & 1 & 1 & 1 & 1 & SIM & R1 -- R6 \\
75 \bottomrule
76 \end{tabular}
77 \end{table}
78 ""
79
80
81 if __name__ == "__main__":
82     print(gerar_tabela_variaveis_latex())
83     print(gerar_tabela_regras_latex())
84     print(gerar_tabela_cenarios_latex())

```

Apêndice G — Declaração de Uso de Inteligência Artificial

Este trabalho acadêmico contou com o uso de ferramentas de Inteligência Artificial generativa (*Anti-gravity/Modelos de Linguagem Avançados*) exclusivamente como apoio instrumental, em conformidade com as diretrizes da CAPES (Brasil) para transparência no uso de IA em produções acadêmicas. A utilização deu-se em dois eixos:

1. **Engenharia de Software Assistida:** apoio na automação de pipelines, refatoração de código limpo (*clean code*), geração de *boilerplates* para testes de contrato (TDD) e formatação de scripts $\text{\LaTeX}$.
2. **Revisão Formal:** apoio à revisão ortográfica, à estruturação de tópicos e à adequação do texto às normas da ABNT.

Ressalta-se que a Inteligência Artificial **não foi utilizada como autora do trabalho**, nem como substituta da elaboração intelectual do discente. Todas as escolhas metodológicas, desenho arquitetural, engenharia de *guardrails*, concepção das regras de negócio (R1–R6), interpretações teóricas e conclusões acadêmicas são de inteira responsabilidade do autor humano. A formulação do modelo lógico proposicional e as salvaguardas de conformidade com a LGPD e o ECA são concepções exclusivamente humanas.

O conteúdo submetido ao apoio da ferramenta passou por revisão crítica, conferência minuciosa (via *quality gates* e testes de contrato) e validação humana, com o objetivo de assegurar a correção das informações, a coerência argumentativa, a conformidade com as regras da ABNT e a adequação acadêmica do texto final.

Assim, o uso da IA neste trabalho teve caráter estritamente instrumental, limitado ao suporte arquitetural de *software*, linguístico, estrutural e normativo, **sem transferência de autoria ou de responsabilidade acadêmica**.